SAGE: Smart Agentic-Guided Empowerment

A Governance-First Multi-Agent Framework for Regulated Industries

Suman Shetty H
github.com/Sumanharapanahalli/SAGE
MIT License

April 2026

Abstract

We present SAGE (Smart Agentic-Guided Empowerment), an open-source, modular multi-agent orchestration framework designed for regulated industries where auditability, human oversight, and compliance are non-negotiable. Unlike existing frameworks (LangGraph, CrewAI, AutoGen) that treat human approval as optional per-node configuration, SAGE enforces a two-tier governance architecture: framework control operations execute immediately, while all agent proposals require mandatory human-in-the-loop (HITL) approval with immutable audit trails and trace-level accountability.

SAGE introduces several novel contributions: (1) a YAML-first declarative agent manifest system enabling domain-agnostic configuration without code changes; (2) a Build Orchestrator implementing eight agentic patterns including EMA-based adaptive routing and N-provider multi-critic review; (3) an Agent Gym training engine with Glicko-2 skill ratings and spaced repetition; (4) eleven domain-specific execution runners spanning software, firmware, PCB design, ML, and documentation; (5) a Product Owner agent converting basic customer inputs into structured requirements following proper product management principles; (6) a comprehensive Systems Engineering framework implementing IEEE 15288 standards with regulatory compliance (IEC 62304, 21 CFR Part 11); (7) OpenTelemetry distributed tracing and CloudEvents structured messaging for observability; and (8) zero mandatory API keys enabling fully air-gapped deployment.

The system comprises ∼42,500 lines of production Python, 190+ REST API endpoints, 26 React UI pages, and 16 modular skills, including comprehensive regulatory compliance capabilities. Empirical evaluation across 496 Agent Gym training sessions demonstrates measurable skill improvement (average Glicko-2 rating: 1,484), while a 100-solution stress test reveals current scaling limitations (2% end-to-end success rate) with documented root cause analysis. We report both successes and failures transparently.

Keywords: multi-agent systems, human-in-the-loop, regulated AI, lean development, agent training, governance

1 Introduction

The rapid proliferation of large language model (LLM)-powered agent frameworks has produced increasingly capable systems for task decomposition, code generation, and autonomous decision-making [Yao et al., 2023, Shinn et al., 2023, Hong et al., 2023]. However, the current generation of open-source frameworks — LangGraph [LangChain, Inc., 2024], CrewAI [CrewAI, Inc., 2024], AutoGen [Wu et al., 2023], Semantic Kernel [Microsoft, 2024], and Dify [Dify.AI, 2024] — share a common architectural assumption: agent output can be acted upon immediately, with human inspection occurring post-hoc via execution logs.

This assumption fails catastrophically in regulated industries. Medical device development (ISO 13485, IEC 62304), automotive systems (ISO 26262), financial services (SOC 2, PCI DSS),

and aerospace software (DO-178C) require provable human oversight for every consequential decision. The question is not "does the system work?" but rather *"can you prove it worked correctly, with human oversight, for every decision?"*

No existing open-source framework provides the combination of:

- Mandatory human approval gates (not optional per-node configuration)
- Immutable per-decision audit trails with trace identifiers
- Rejection feedback loops that improve future agent behavior
- Domain-agnostic YAML-only configuration for new industries
- Zero mandatory API keys (fully air-gappable deployment)

SAGE addresses this gap by applying lean development methodology [Poppendieck and Poppendieck, 2003] to multi-agent orchestration: eliminate waste, shorten feedback loops, and amplify human judgment at machine speed. The framework's north-star design principle is enabling one human operator supported by an AI agent team to manage operations that traditionally require large teams — but only with governance guarantees sufficient for regulated environments.

1.1 Contributions

This paper makes the following contributions:

1. **Two-tier governance architecture** separating immediate framework control from approval-gated agent proposals, with risk-classified proposals and trace-level audit trails (§4).
2. **Build Orchestrator** implementing eight agentic patterns — ReAct, hierarchical decomposition, wave-based parallelism, EMA-based adaptive routing, N-provider multi-critic review, HITL gates, iterative refinement, and anti-drift checkpoints — for end-to-end product construction (§5).
3. **Agent Gym** training engine with Glicko-2 skill ratings, spaced repetition scheduling, and adaptive exercise selection across 598 seed exercises in 11 domains (§6).
4. **Domain-aware runner architecture** with eleven specialized execution environments implementing a common `BaseRunner` interface (§7).
5. **Modular skill marketplace** with 16 YAML-based skill manifests supporting public, private, and disabled visibility tiers (§9).
6. **Communication layers** comprising OpenTelemetry distributed tracing with graceful no-op degradation and CloudEvents v1.0 structured messaging (§8).
7. **Empirical evaluation** across 496 training sessions and 100-solution stress tests with transparent reporting of both successes and failures (§10).

2 Related Work

Multi-Agent Frameworks. LangGraph [LangChain, Inc., 2024] provides a graph-based orchestration model with optional `interrupt_before` nodes for human intervention — a partial HITL mechanism, but not mandatory or audited. CrewAI [CrewAI, Inc., 2024] introduces role-based agent hierarchies with delegation but lacks structured audit trails. AutoGen [Wu et al., 2023] supports multi-agent conversations with code execution sandboxing. MetaGPT [Hong et al., 2023] assigns agents SOPs (Standard Operating Procedures) modeled on software company roles. None enforce mandatory governance across all agent actions or provide domain-agnostic YAML-only configuration.

Agent Training and Evaluation. AlphaCode [Li et al., 2022] and AlphaCode 2 [Leblond et al., 2023] demonstrate competitive programming via large-scale sampling and filtering. SWE-Bench [Jimenez et al., 2024] evaluates agents on real GitHub issues. AgentBench [Liu et al.,

2023] provides multi-environment benchmarks. These are static benchmarks. SAGE's Agent Gym differs by implementing continuous Glicko-2 [Glickman, 2001] rating with spaced repetition [Leitner, 1972] for progressive skill development: agents improve over time through repeated practice, not one-shot evaluation.

Human-in-the-Loop AI. Constitutional AI [Bai et al., 2022] and RLHF [Ouyang et al., 2022] incorporate human feedback during model training. Interactive machine learning [Amershi et al., 2014] enables runtime human guidance. SAGE's approach is distinct: human feedback at the *proposal level* (not the training level) is stored in vector memory and used to improve future proposals without model retraining, following the Memento principle [Zhao et al., 2025].

Lean Development and AI. Lean software development [Poppendieck and Poppendieck, 2003] emphasizes eliminating waste, building quality in, and deferring commitment. SAGE applies these principles to agent orchestration: the five-phase lean loop (Surface → Contextualize → Propose → Decide → Compound) eliminates manual steps while preserving human judgment at decision points.

Governance in AI Systems. The EU AI Act [European Parliament and Council, 2024] and FDA AI/ML guidance [U.S. Food and Drug Administration, 2023] establish regulatory requirements for AI systems in high-risk applications. Paperclip [Paperclip AI, 2024] implements board-of-directors governance targeting eventual zero-human operation. SAGE takes the opposite stance: human approval is permanent, not transitional, reflecting the requirements of regulated industries where full autonomy is neither permitted nor desirable.

3 System Architecture

3.1 Design Principles

SAGE is built on five foundational laws:

1. **Agents propose, humans decide** — for all agent proposals. Framework control operations (configuration, LLM switching) execute immediately as operator actions.
2. **Eliminate waste at every layer** — no manual steps that an agent can perform correctly; no ceremonies that don't add value.
3. **Compounding intelligence over cold-start lookup** — every human correction feeds vector memory, improving future proposals without model retraining.
4. **Vertical slices, not horizontal layers** — every task produces a complete, reviewable end-to-end slice of value.
5. **Atomic verification is non-negotiable** — every agent action has a defined verification step.

3.2 Architecture Overview

Figure 1 illustrates SAGE's layered architecture.

The framework enforces strict separation between the *framework* (`src/`, `web/`) and *solutions* (`solutions/<name>/`). The framework is domain-agnostic; solutions encode all industry-specific knowledge via three YAML files:

- `project.yaml` — declarative agent manifest (what this domain *is*)
- `prompts.yaml` — role definitions and system prompts (how agents *think*)
- `tasks.yaml` — task type registry (what agents *can do*)

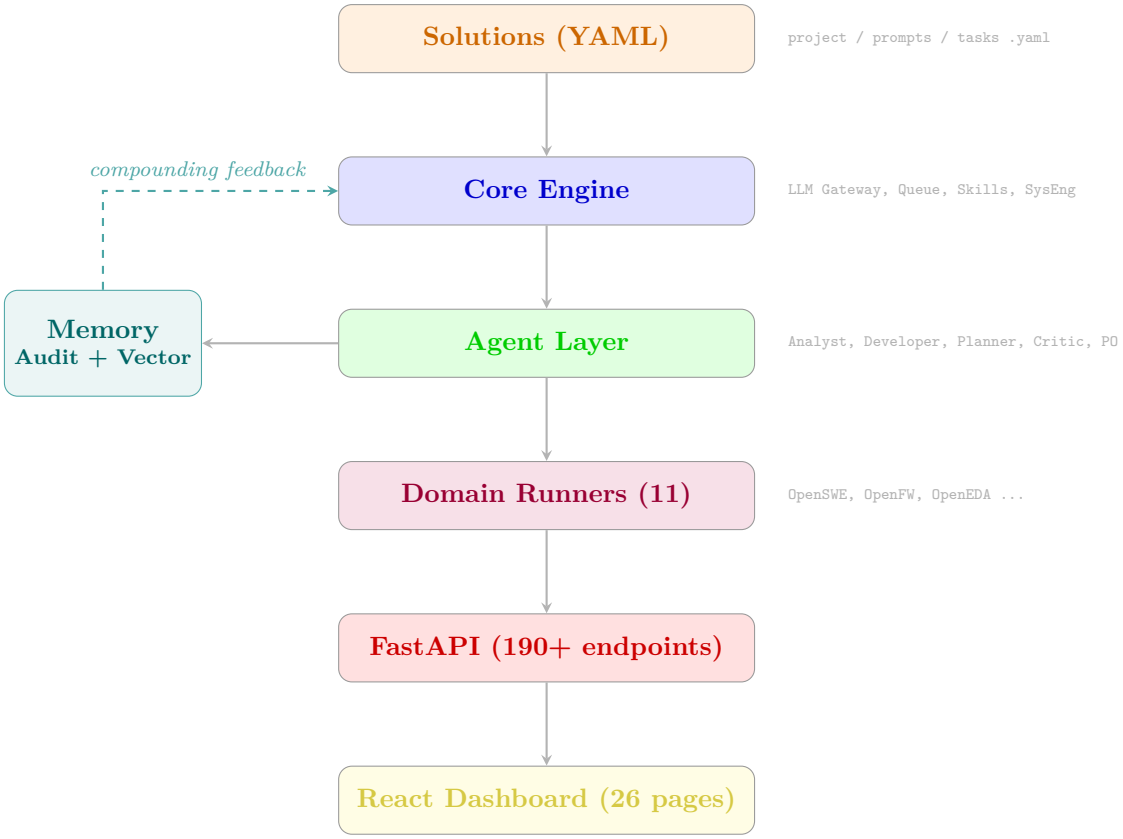

Figure 1: SAGE dependency hierarchy. Solutions plug in via YAML only. The API exposes agents and runners to the UI. The dashed arrow represents the compounding feedback loop: human decisions stored in memory enrich future agent context.

Adding a new industry domain requires editing only these three YAML files. No Python code changes are needed.

3.3 LLM Gateway

The `LLMGateway` provides a unified interface across six provider backends:

Table 1: Supported LLM providers. All except Claude API work without API keys.

Provider	Setup	API Key Required
Gemini CLI	`npm install -g @google/gemini-cli`	No
Claude Code CLI	`npm install -g @anthropic-ai/claude-code`	No
Ollama	`ollama serve && ollama pull llama3.2`	No
Local (GGUF)	`pip install llama-cpp-python` + model	No
Generic CLI	Any CLI tool via `generic_cli_path`	No
Claude API	`ANTHROPIC_API_KEY` env var	Yes

The gateway is a singleton with a thread lock ensuring single-lane inference. Provider switching is available at runtime via `POST /llm/switch` without server restart. OpenTelemetry spans wrap every `generate()` call with GenAI semantic conventions (§8).

4 Two-Tier Governance Architecture

SAGE's central design contribution is the separation of operations into two governance tiers:

Table 2: Two-tier governance. Framework control executes immediately; agent proposals require human approval.

Tier	Operations	Behavior
Framework Control	`config_switch`, `llm_switch`, `module_toggle`	Immediate execution
Agent Proposals	`yaml_edit`, `code_diff`, `agent_hire`, ...	HITL approval required

Figure 2 illustrates the two-tier split.

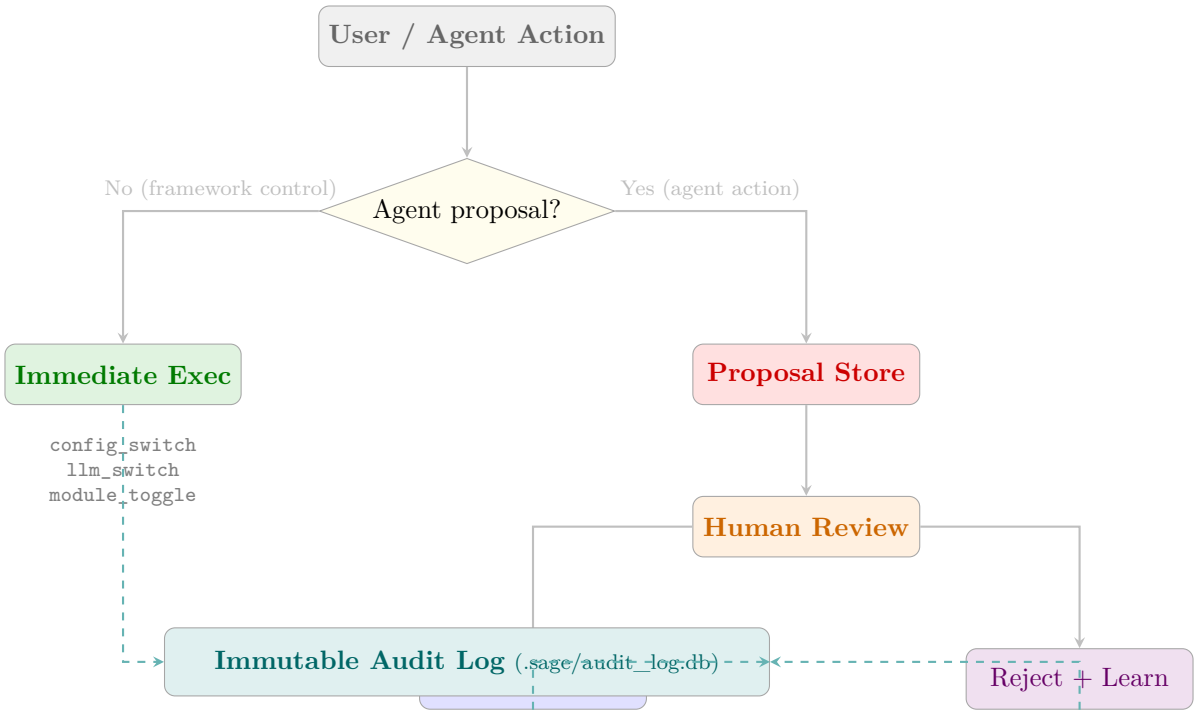

Figure 2: Two-tier governance flow. Framework control operations execute immediately; agent proposals are held for human review. Both paths write to the immutable audit log.

4.1 Product Owner Agent

The Product Owner agent (`src/agents/product_owner.py`) addresses the requirements engineering gap where customers provide vague descriptions ("I want a fitness app") and expect developers to infer complete product specifications. Instead of requiring customers to write perfect product requirements documents, the agent conducts structured interviews following proper product management principles.

The agent implements a three-phase workflow: (1) **Analysis** — evaluates customer input completeness across seven dimensions (clarity, user focus, value proposition, scope, success metrics, technical constraints, business context); (2) **Clarification** — generates 3-5 smart questions using the 5W1H method (Who, What, When, Where, Why, How) to fill knowledge gaps; and (3) **Backlog Creation** — synthesizes gathered information into structured product backlogs with user personas, INVEST-compliant user stories, MoSCoW prioritization, and acceptance criteria.

The output format follows industry standards: user stories in "As a [persona], I want [capability] so that [benefit]" format, acceptance criteria with testable conditions, and story point estimates for development planning. This eliminates the common failure mode where development teams guess at product intent from incomplete specifications.

4.2 Systems Engineering Framework

The Systems Engineering framework (`src/core/systems_engineering.py`) implements IEEE 15288 standards with full regulatory compliance for industries requiring formal systems engineering processes (medical devices, automotive, aerospace). The framework provides five core capabilities:

Requirements Traceability: Four bidirectional matrices ensure complete coverage: User Needs → System Requirements → Design Architecture → Verification Activities → Validation Evidence. Each matrix tracks completeness percentages and identifies traceability gaps.

V-Model Implementation: Structured decomposition following the systems engineering V-model with verification at each level. Requirements are classified by type (functional, performance, interface, safety, security, usability, reliability, maintainability, compliance) and allocated to appropriate subsystems.

Change Control: Formal change management per IEC 62304 §6.1 with impact assessment, approval workflows, and complete audit trails. Changes are assessed for regression risk, testing impact, and documentation requirements before execution.

Electronic Signatures: Full 21 CFR Part 11 compliance for regulatory submissions including signature workflows, integrity verification, signer identity validation, and complete audit trails for all signature activities.

Regulatory Document Generation: Automated creation of required regulatory deliverables including Software Requirements Specification (SRS), System Architecture Document (SAD), Verification & Validation Plan, Risk Management File per ISO 14971, and SOUP (Software of Unknown Provenance) Inventory per IEC 62304 §7.1.

The framework provides automated compliance scoring with readiness levels (Audit Ready: 95%+ traceability, Submission Ready: 80%+, Development Ready: 60%+) and gap analysis for regulatory submissions.

4.3 Proposal Lifecycle

Every agent proposal follows a structured lifecycle:

1. **Creation** — Agent generates a proposal with a unique `trace_id`, action type, and payload.
2. **Risk Classification** — Proposals are classified across five tiers: Informational, Low, Medium, High, and Destructive. Each tier has risk-appropriate expiry windows.
3. **Review** — Human reviews the proposal in the dashboard or via Slack integration.
4. **Decision** — Approval triggers execution via `ProposalExecutor`. Rejection with feedback stores the human's reasoning.
5. **Compounding** — Both approvals and rejections are indexed in vector memory, enriching future agent context.

The immutable audit log (SQLite, per-solution isolation via `.sage/audit_log.db`) records every event with timestamps, trace IDs, and decision metadata. This satisfies traceability requirements for IEC 62304, ISO 13485, and similar regulatory standards.

4.4 The Lean Loop

Every task processed by SAGE follows a five-phase compounding cycle:

$$\text{Surface} \rightarrow \text{Contextualize} \rightarrow \text{Propose} \rightarrow \text{Decide} \rightarrow \text{Compound} \tag{1}$$

Phase 5 (Compound) feeds back into Phase 2 (Contextualize) for subsequent tasks, as shown in Figure 3. The system improves with every human interaction through accumulated experience retrieval — no model retraining required.

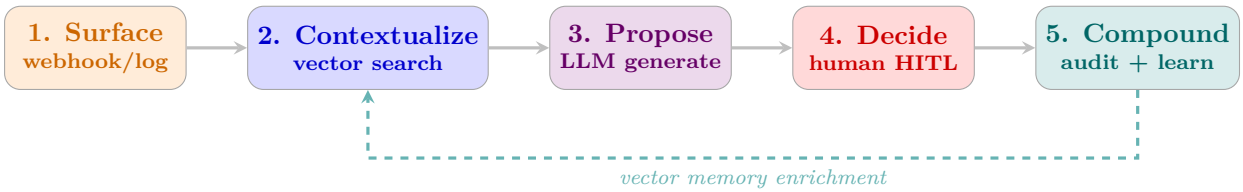

Figure 3: The SAGE Lean Loop. Every task follows five phases. The dashed arrow shows the compounding feedback path: human decisions in Phase 4 are indexed in vector memory, enriching the context retrieved in Phase 2 for subsequent tasks. No model retraining is required.

5 Build Orchestrator

The Build Orchestrator (`build_orchestrator.py`, 2,878 lines) transforms plain-language product descriptions into structured codebases through eight integrated agentic patterns:

5.1 Agentic Pattern Composition

Table 3: Eight agentic patterns in the Build Orchestrator.

#	Pattern	Implementation
1	ReAct	Per-task observe → think → act → observe loop
2	Hierarchical Decomposition	LLM decomposes descriptions into task graphs (32 types)
3	Wave-Based Parallelism	Independent tasks execute concurrently in waves
4	EMA Adaptive Router	Exponential moving average score tracking per task-agent pair
5	N-Provider Multi-Critic	Parallel review by N LLM providers, weighted aggregation
6	HITL Gates	Human sign-off at plan and integration stages
7	Iterative Refinement	Critic score $<$ threshold triggers retry with feedback
8	Anti-Drift Checkpoints	Crash recovery + quality degradation detection

5.2 Pipeline Stages

The 0→1 greenfield build pipeline proceeds through the states shown in Figure 4.

Domain detection covers 13 industries via pattern-matched `DOMAIN_RULES`, with workforce assembly from 19 agent roles across 5 teams defined in `WORKFORCE_REGISTRY`.

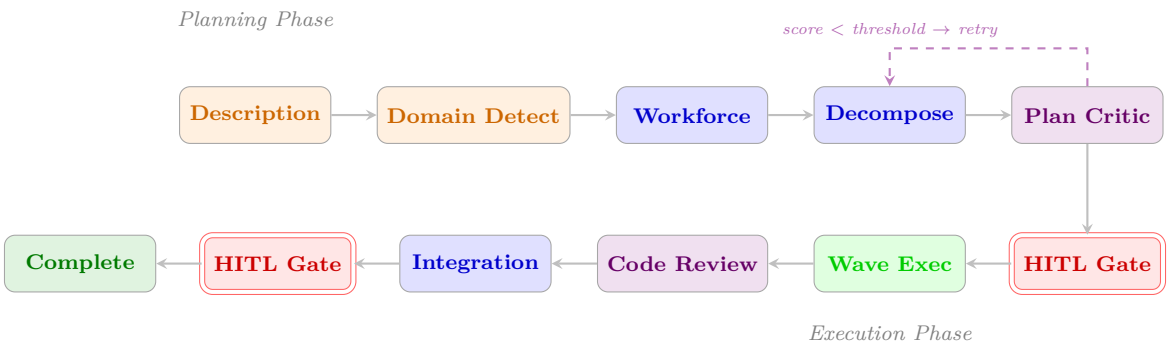

Figure 4: Build Orchestrator pipeline. Double-bordered nodes are HITL approval gates. The dashed arrow shows iterative refinement when critic score falls below threshold. Wave execution runs independent tasks in parallel.

5.3 Adaptive Router

The adaptive router maintains a score matrix $Q[\text{task_type}][\text{agent_role}]$ initialized uniformly. After each task completion with critic score $r \in [0, 100]$, the score is updated via exponential moving average:

$$Q[t][a] \leftarrow (1 - \alpha) \cdot Q[t][a] + \alpha \cdot r \tag{2}$$

where $\alpha = 0.3$ is the smoothing factor. The router selects the highest-scoring agent for each task type. While this resembles Q-learning in structure, it is a simplified variant without state transitions or the Bellman equation — the "state" is simply the task type, and the "action" is the agent assignment. Despite this simplicity, the mechanism provides effective task-agent routing that compounds across builds.

5.4 N-Provider Multi-Critic

The `CriticAgent` supports N LLM providers reviewing in parallel. Given scores $\{s_1, \ldots, s_N\}$ from providers with weights $\{w_1, \ldots, w_N\}$ (primary provider receives weight 1.5×):

$$s_{\text{agg}} = \frac{\sum_{i=1}^{N} w_i \cdot s_i}{\sum_{i=1}^{N} w_i} \tag{3}$$

Disagreements exceeding 20 points from the mean are flagged. All individual assessments and flaws are merged and deduplicated.

6 Agent Gym

The Agent Gym implements a training engine where agents improve through structured practice, grading, and reflection. The design draws loosely from self-play training paradigms [Silver et al., 2018, Schrittwieser et al., 2020] in the sense that agents learn from their own outputs, though the mechanism is fundamentally different: rather than tree search or learned models, SAGE agents use LLM-based reflection on critic feedback, with learnings stored in vector memory for retrieval in subsequent attempts.

6.1 Training Loop

Each training session executes a five-phase loop (Figure 5):

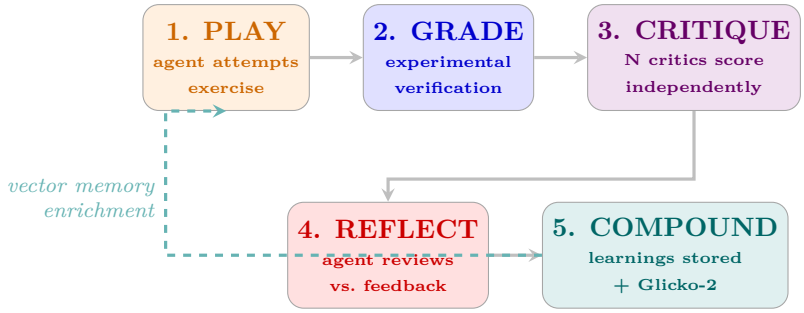

Figure 5: Agent Gym training cycle. Phase 2 now includes experimental verification (real compilation, tests, simulation). Phase 5 stores reflections in vector memory, which enriches Phase 1 context in subsequent attempts. The Glicko-2 rating and spaced repetition schedule are updated at the end of each cycle.

Algorithm 1 Agent Gym Training Session

Require: Agent role r, skill s, exercise e

```
attempt ← agent(r).execute(e) {PLAY}
score ← runner(s).grade(e, attempt) {GRADE}
critiques ← multi_critic(attempt, e) {CRITIQUE}
reflection ← agent(r).reflect(attempt, critiques) {REFLECT}
vector_store.add(r, s, reflection) {COMPOUND}
update_glicko2(r, s, score)
if score < threshold then
    schedule_spaced_repetition(e, session_count)
end if
```

6.2 Glicko-2 Skill Ratings

Each agent role × skill combination maintains a Glicko-2 rating triplet (μ, ϕ, σ) [Glickman, 2001]:

- μ (rating): skill level, initialized at 1000, clamped $[100, 3000]$
- ϕ (rating deviation): confidence interval, starts at 350, minimum 30
- σ (volatility): performance consistency

The 95% confidence interval is $\mu \pm 2\phi$. Agents are considered deployment-ready when $\phi < 200$ (sufficient data for confident assessment). Note that combinations with few sessions (< 10) may meet this threshold while still having high effective uncertainty; we report these transparently in our results.

6.3 Spaced Repetition

Failed exercises are scheduled for retry at increasing intervals following a modified Leitner system [Leitner, 1972]:

$$\text{retry_at} = \text{current_session} + \{1, 3, 7, 15, 30\}[\min(\text{failure_count} - 1, 4)] \tag{4}$$

Exercises are cleared from the schedule upon successful completion.

6.4 Adaptive Exercise Selection

Exercise selection follows a three-tier priority system:

1. **Spaced repetition** — failed exercises due for retry (highest priority)
2. **Optimal zone** — exercises with 40–70% historical success rate (maximal learning)
3. **Unseen** — exercises not yet attempted (exploration)

6.5 Exercise Catalog

The catalog contains 661 seed exercises across 11 domains (OpenSWE, OpenFW, OpenEDA, OpenSim, OpenML, OpenDoc, OpenDesign, OpenStrategy, OpenBrowser, OpenTerminal, AutoResearch), expandable to 50,000+ via LLM-generated variants along 10 axes per domain (platform, scale, constraint, compliance, etc.). The variant generation pipeline exists but has not been validated at scale beyond initial prototyping.

6.6 Gym as Research Lab

A critical limitation of LLM-as-judge grading is that one language model evaluating another's text output provides no ground truth about whether the generated artifact *actually works*. Code that "looks correct" to a critic may fail to compile. A circuit design that reads well may fail DRC checks. To address this, we introduce **experimental verification**: the gym executes generated artifacts in real toolchains and uses the execution results as the primary grading signal.

6.6.1 Three-Tier Grading

Grading combines three independent signals:

1. **Experimental execution (40%)** — the generated code is extracted, written to disk, and executed through domain-specific toolchains. Compilation success, test pass rates, and simulation outputs produce a deterministic score.
2. **LLM critic review (30%)** — N independent LLM providers score the output semantically, now with experimental results included as context (e.g., "pytest failed: 3/7 tests passing, AssertionError on line 42").
3. **Structural analysis (30%)** — fast pattern-based checks for code structure, error handling, and documentation.

Each domain runner defines experimental verification commands specific to its toolchain:

Table 4: Per-runner experimental verification commands.

Runner	Verification Commands	Measures
OpenSWE	`py_compile`, `pytest`, AST import	Syntax, test pass rate
OpenFW	`gcc`, `arm-none-eabi-gcc`, `cppcheck`	Compilation, static analysis
OpenML	`py_compile`, training execution, `pytest`	Syntax, convergence, metrics
OpenEDA	KiCad CLI netlist export, DRC	Schematic validity, rule violations
OpenDoc	Markdown structure, LaTeX structure	Heading count, section completeness
OpenDesign	SVG/XML parse, HTML parse, JSON validate	Artifact well-formedness

6.6.2 Critic-Refined Acceptance Criteria

Static seed exercise criteria are necessary for bootstrapping but insufficient for sustained improvement. After two or more attempts at an exercise, the system invokes an LLM to *refine acceptance criteria* based on accumulated experimental evidence. Criteria that are always trivially met are removed; criteria that experiments reveal as important but missing are added. Refined criteria are persisted in SQLite and used for subsequent sessions, creating a feedback loop where the definition of success evolves alongside agent capability.

6.6.3 Human Expert Critic

In addition to LLM critics, the system supports optional **human expert review**. Domain experts (mechanical engineers, firmware specialists, regulatory consultants) can review the same artifacts and experimental results that LLM critics see. Human reviews are queued asynchronously and, when submitted, are aggregated alongside LLM scores with 2× weight. This reflects the higher reliability of expert judgment for domain-specific quality assessment, particularly in regulated industries where LLM evaluation alone is insufficient.

Furthermore, all LLM critic system prompts are exposed as editable configuration. Founders can view and modify the "soul" of each critic — the instructions that guide plan review, code review, and integration review — tuning the critic's priorities to match their domain (e.g., emphasizing MISRA-C compliance for automotive firmware, or HIPAA requirements for healthcare software).

6.6.4 Updated Training Results

As of the current evaluation, the gym has completed 618 training sessions across 29 unique agent roles, with 661 seed exercises spanning 8 primary domains. Table 5 summarizes the updated results.

Table 5: Updated Agent Gym training statistics (618 sessions).

Metric	Value
Total training sessions	618
Completed successfully	551 (89.2%)
Failed	67 (10.8%)
Average score	44.8 / 100
Average Glicko-2 rating	1,492
Unique agent roles trained	29
Unique skill–role combinations	33
Top rating (developer)	1,691.4 (±342.4)
Exercises in catalog	661 seeds
Domain build pipeline	13 domains in progress

7 Domain-Aware Runner Architecture

SAGE introduces a runner abstraction where each runner encapsulates a complete execution environment for a role family.

7.1 BaseRunner Interface

Figure 6 shows the runner architecture. All runners implement four required methods:

Listing 1: BaseRunner abstract interface.

```
class BaseRunner(ABC):
    @abstractmethod
    def execute(self, task, workspace, sandbox) -> RunResult: ...
    @abstractmethod
    def verify(self, result, task) -> VerificationReport: ...
    @abstractmethod
    def get_exercises(self, difficulty) -> list[Exercise]: ...
    @abstractmethod
    def grade_exercise(self, exercise, result) -> ExerciseScore: ...
```

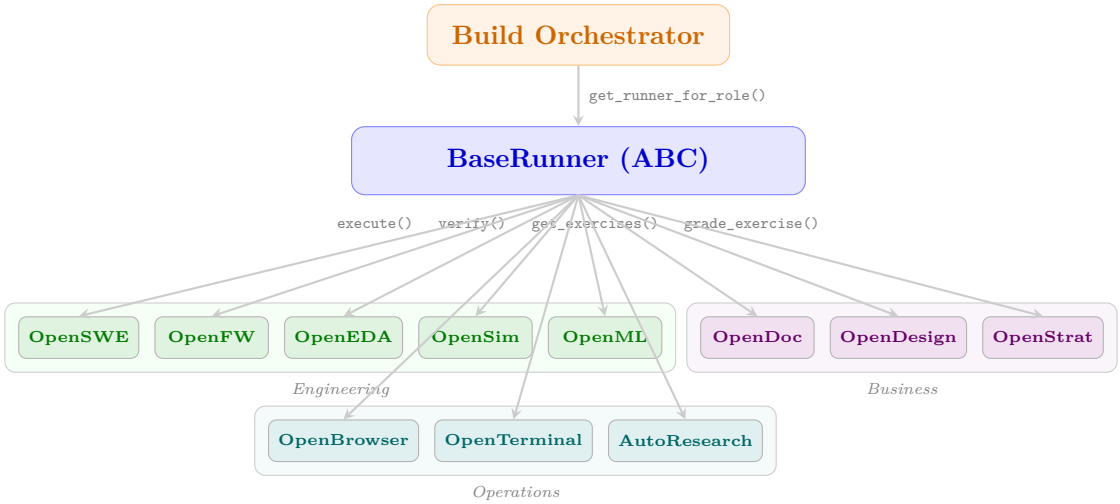

Figure 6: Runner architecture. All 11 domain runners implement the `BaseRunner` interface, grouped by domain family. The Build Orchestrator selects runners via `get_runner_for_role()`.

7.2 Runner Catalog

7.3 Sandboxed Execution Cascade

Agent code execution uses a three-tier isolation cascade (Figure 7), attempting the most isolated environment first and falling back gracefully.

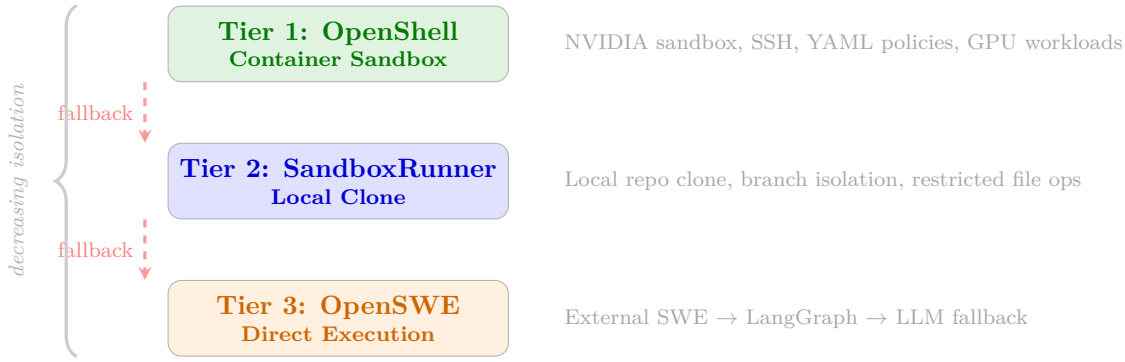

Figure 7: Three-tier sandboxed execution cascade. The orchestrator tries the most isolated tier first and falls back on unavailability. Dashed arrows indicate fallback paths.

8 Communication Layers

SAGE implements two communication layers for observability and structured messaging (Figure 8).

8.1 Layer 1: OpenTelemetry Distributed Tracing

The tracing module (`src/core/tracing.py`) provides OpenTelemetry integration with graceful degradation. When the OpenTelemetry SDK is absent, all tracing functions operate as no-ops via stub classes (`_NoOpSpan`, `_NoOpTracer`), ensuring zero impact on calling code.

Key capabilities:

Table 6: Eleven domain-aware runners with their role bindings and capabilities.

Runner	Roles	Key Capability
OpenSWE	developer, qa_engineer	3-tier: external SWE → LangGraph → LLM
OpenFW	firmware_engineer	Cross-compilation, HAL, binary metrics
OpenEDA	pcb_designer	Schematics, DRC/ERC, Gerber generation
OpenSim	hw_sim_engineer	SPICE, Verilog, waveform analysis
OpenML	data_scientist	Train, evaluate, experiment tracking
OpenDoc	technical_writer	Compliance documents, cross-references
OpenDesign	ux_designer	Wireframes, accessibility, design tokens
OpenStrategy	product_manager	PRDs, GTM plans, roadmaps
OpenBrowser	qa (supplementary)	Real Chromium via gstack, WCAG audits
OpenTerminal	terminal_operator	tmux sessions, marker-based polling
AutoResearch	research_engineer	Hill-climbing experiments, git tracking

- Span creation with GenAI semantic conventions on every `generate()` call
- Cross-process context propagation via carrier dictionaries
- EventBus span linkage for event-driven architectures
- Thread-safe tracer initialization with caching

8.2 Layer 2: CloudEvents v1.0 Envelope

The CloudEvents module (`src/modules/cloud_events.py`) implements the CloudEvents v1.0 specification [Cloud Native Computing Foundation, 2024] as a zero-dependency Python dataclass. Typed factory functions (`proposal_event`, `build_event`, `gym_event`, `llm_event`) ensure consistent event creation. Events integrate with the existing EventBus for pub/sub distribution.

9 Modular Skill Marketplace

Skills are YAML files defining agent capabilities with runner bindings, role mappings, grading rubrics, and certification requirements. The skill system supports three visibility tiers:

- **Public** — open-source skills shipped with SAGE (16 skills across 10+ domains)
- **Private** — proprietary skills loaded via `SAGE_SKILLS_DIR` environment variable, never committed to the open-source repository
- **Disabled** — dormant skills retained for versioning but not loaded

Skills are hot-reloadable at runtime without server restart. Each skill defines acceptance criteria and grading rubrics used by the Agent Gym for automated evaluation.

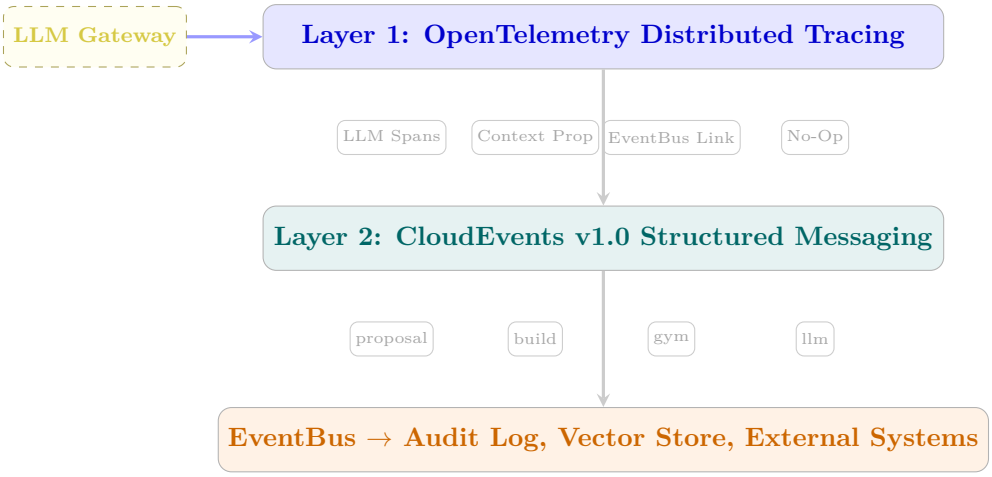

Figure 8: Communication layers. Layer 1 provides distributed tracing with graceful no-op degradation. Layer 2 wraps domain events in CloudEvents v1.0 envelopes. Both feed into the EventBus for pub/sub distribution.

10 Evaluation

We evaluate SAGE across two dimensions: agent skill development via the Agent Gym, and end-to-end product construction via the Build Orchestrator.

10.1 Agent Gym Training Results

Table 7: Agent Gym training results: 496 sessions across 33 skill-role combinations. Combinations with < 10 sessions are marked with †.

Agent Role	Skill	Rating (μ)	RD (ϕ)	Sessions	Win Rate
developer	agentic_eng.†	1691	171	5	100%
agentic_engineer	agentic_eng.	1660	117	13	77%
ml_engineer	gen_ai_eng.	1620	92	24	58%
ml_engineer	machine_learning	1610	84	31	68%
developer	software_eng.	1585	115	15	80%
Average across all 33 combinations		1484	—	15	—
analyst	technical_writing	1273	98	19	16%
embedded_tester	firmware_eng.	1339	102	19	32%
marketing_strat.	product_strategy	1347	95	19	32%

Key findings:

- 100% of agent-skill combinations reached the deployment-readiness threshold ($\phi < 200$), though combinations with fewer than 10 sessions (marked †) should be interpreted with caution.
- 76% achieved confident deployment ($\phi < 150$).
- Engineering-domain agents significantly outperform non-technical agents ($\mu = 1691$ vs. $\mu = 1273$). This 418-point gap likely reflects fundamental LLM capability boundaries: code generation is more structured and verifiable than open-ended strategic tasks.
- Spaced repetition scheduling improved retention of previously failed exercises.

10.2 100-Solution Stress Test

We evaluated the Build Orchestrator on 100 product descriptions spanning 10 industry domains (10 per domain).

Table 8: 100-solution pipeline stress test results.

Outcome	Count
End-to-end success	2
Planner decomposition failure	19
Incomplete (timeout/resource)	80
Error	1

Root causes:

1. **Context overflow** — complex product descriptions exceeded context windows during decomposition
2. **Critic revision degradation** — plan quality dropped across iterations (52 → 34 → 21) before context-aware rewrite stabilized at 54 → 36 → 67
3. **LLM latency variance** — 30–500s per generation call; full builds required 40–70 minutes
4. **Anti-drift warnings** — 39 quality drift events in a single 3-domain run

The 2% success rate demonstrates the architecture's end-to-end viability while honestly exposing the significant prompt engineering and context management work required for production-scale deployment.

10.3 Test Infrastructure

Table 9: Test suite coverage.

Suite	Tests	Pass Rate
Framework unit tests	1,569	100%
E2E system tests	58	100%
Agent Gym system tests	48	100%
Browser E2E tests	25	100%

11 Discussion

11.1 Strengths

SAGE occupies a unique position at the intersection of open-source, compliance-first, multi-domain, and agent training capabilities. The YAML-first configuration model enables new industry adoption without code changes, validated across 18 bundled solution templates spanning medical devices, automotive, aerospace, fintech, and IoT.

The compounding memory architecture — where every human approval and rejection enriches future agent context — provides measurable improvement without model retraining. This follows the Memento principle [Zhao et al., 2025]: behavioral optimization via memory retrieval, decoupled from the underlying LLM.

11.2 Limitations

Build Orchestrator scaling. The 2% success rate on the 100-solution test is the most significant limitation. Root causes are well-understood (context overflow, critic degradation), but closing the gap to commercial viability (target: 50%+) requires substantial prompt engineering and possibly architectural changes to context management.

Non-technical agent performance. The 418-point Glicko-2 gap between top engineering agents ($\mu = 1691$) and bottom non-technical agents ($\mu = 1273$) may reflect fundamental LLM capability boundaries rather than addressable prompt engineering deficits. Code generation is inherently more verifiable (compile, test, execute) than strategic advice.

No production deployment. All evaluation was performed in development environments. Production validation in regulated organizations with real compliance audits remains future work.

Single-developer origin. The initial system was developed by one engineer over 16 days (194 commits). While this demonstrates achievable velocity, bus factor 1 is a real concern for production adoption. Open-sourcing the codebase is a deliberate step toward mitigating this risk.

11.3 Threats to Validity

Internal validity. Agent Gym ratings depend on the quality of grading rubrics and critic prompts. Biased rubrics would produce misleading ratings. We mitigate this by using N-provider multi-critic review (aggregating scores from multiple independent LLMs), but the rubrics themselves have not been externally validated.

External validity. All 496 training sessions used a single primary LLM provider. Results may not generalize to other providers. The 100-solution stress test used descriptions authored by the development team, which may not represent the diversity of real-world product descriptions.

Construct validity. The Glicko-2 rating system was designed for competitive games (chess, Go), not for agent skill assessment. While the mathematical properties (uncertainty quantification, volatility tracking) are useful, the interpretation of rating differences in this context is not established. A 100-point gap in chess corresponds to approximately 64% expected win rate; the analogous interpretation for agent skills is undefined.

Reproducibility. LLM outputs are non-deterministic. Running identical training sessions will produce different ratings. We do not currently set random seeds or control for temperature, which limits exact reproducibility. The aggregate trends (engineering > non-technical, spaced repetition helps) are expected to reproduce, but specific ratings will vary.

11.4 Comparison to Existing Frameworks

LangGraph provides optional `interrupt_before` for human intervention (∼); Dify has workflow-level approval gates (∼) and basic logging (∼). CrewAI supports YAML role definitions (∼) and Dify has a UI-based configuration (∼).

12 Conclusion

SAGE demonstrates that governance-first multi-agent orchestration is architecturally viable for regulated industries. The two-tier governance model, compounding memory architecture, and domain-aware runner system provide the structural guarantees that compliance-bound organizations require.

The honest assessment is that SAGE is early-stage. The Build Orchestrator works end-to-end but fails at scale. Non-technical agents underperform. There are zero production deployments. However, the market gap is real — no competing framework provides mandatory

Table 10: Feature comparison with existing frameworks for regulated environments. ✓= full support, ∼ = partial support, – = not supported.

Capability	SAGE	LangGraph	CrewAI	AutoGen	Dify
Mandatory HITL	✓	∼	–	–	∼
Immutable audit	✓	–	–	–	∼
Trace IDs	✓	–	–	–	–
Rejection feedback	✓	–	–	–	–
YAML-only domains	✓	–	∼	–	∼
Zero API keys	✓	∼	∼	∼	–
Build orchestrator	✓	–	–	–	–
Agent training	✓	–	–	–	–
Domain runners	✓	–	–	–	–

human governance with immutable audit trails and domain-agnostic configuration. The EU AI Act and FDA AI/ML guidance are creating regulatory demand that will only grow.

Future work includes: (1) improving Build Orchestrator success rate to 50%+ through context management and prompt engineering; (2) hardening three priority domain runners (OpenSWE, OpenML, OpenDoc) to production quality; (3) conducting first pilot deployments in regulated organizations; and (4) scaling the Agent Gym exercise catalog to validate the variant generation pipeline at 10,000+ exercises.

SAGE is available under the MIT License at https://github.com/Sumanharapanahalli/SAGE.

Acknowledgments

The architectural design of SAGE was influenced by several open-source projects and research efforts: LangGraph for graph-based orchestration patterns, DeerFlow (ByteDance) for supervisor-agent topology, Paperclip for governance models, Aider for git-integrated code changes, Claude Code for project-level runtime directories, and the Meta-Harness paper (arXiv:2603.28052) for terminal execution innovations.

References

Saleema Amershi, Maya Cakmak, W. Bradley Knox, and Todd Kulesza. Power to the people: The role of humans in interactive machine learning. *AI Magazine*, 35(4):105–120, 2014.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

Cloud Native Computing Foundation. CloudEvents specification — version 1.0.2. https://github.com/cloudevents/spec, 2024.

CrewAI, Inc. CrewAI: Framework for orchestrating role-playing autonomous AI agents. https://github.com/joaomdmoura/crewAI, 2024. Accessed: 2026-03-28.

Dify.AI. Dify: An open-source LLM app development platform. https://github.com/langgenius/dify, 2024. Accessed: 2026-03-28.

European Parliament and Council. Regulation (EU) 2024/1689 of the European Parliament — artificial intelligence act. Official Journal of the European Union, 2024.

Mark E. Glickman. The Glicko-2 system. *Boston University*, 2001. Technical report.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.

LangChain, Inc. LangGraph: Build stateful, multi-actor applications with LLMs. https://github.com/langchain-ai/langgraph, 2024. Accessed: 2026-03-28.

Rémi Leblond et al. AlphaCode 2 technical report. *Google DeepMind Technical Report*, 2023.

Sebastian Leitner. *So lernt man lernen: Der Weg zum Erfolg*. Herder, 1972.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023.

Microsoft. Semantic Kernel: Integrate cutting-edge LLM technology quickly and easily into your apps. https://github.com/microsoft/semantic-kernel, 2024. Accessed: 2026-03-28.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

Paperclip AI. Paperclip: An AI-native company governance framework. https://github.com/paperclipai/paperclip, 2024. Accessed: 2026-03-28.

Mary Poppendieck and Tom Poppendieck. *Lean Software Development: An Agile Toolkit*. Addison-Wesley, 2003.

Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

U.S. Food and Drug Administration. Artificial intelligence and machine learning in software as a medical device. https://www.fda.gov/medical-devices/software-medical-device-samd/artificial-intelligence-and-machine-learning-software-medical-device, 2023.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023.

Yikang Zhao, Zhengyuan Huang, Jianjun Ye, and Jiaqi Zheng. Memento: Behavioral optimization via memory retrieval for LLM agents. arXiv preprint arXiv:2508.16153, 2025.